Experiment 9: Delta Rule

Objective

To implement the Delta Rule for adjusting the weights of a single neuron in order to minimize the error between predicted and target outputs.

Theory

The Delta Rule, also known as the Widrow-Hoff learning rule, is used to train a single-layer neural network. It updates the weights based on the error between the predicted output and the actual target output.

The weight update rule is given by:

$$\Delta w = \eta(t - y)x$$

$$w_{\text{new}} = w_{\text{old}} + \Delta w$$

Where:

- η = Learning rate
- t = Target output
- y = Predicted output
- x = Input

This rule helps in minimizing the Mean Squared Error (MSE) and is the foundation of gradient descent learning.

Program

```
import numpy as np

# Input (with bias term included)
X = np.array([
    [1, 0.5, 1.5],    # sample 1
    [1, 1.0, -0.5],   # sample 2
    [1, -1.5, 2.0]    # sample 3
])

# Target outputs
T = np.array([1, -1, 1])

# Initialize weights (including bias weight)
W = np.random.randn(3)

# Learning rate
eta = 0.1

# Training loop
for epoch in range(50):
    total_error = 0

    for i in range(len(X)):
        x = X[i]
        t = T[i]

        # Output (linear neuron)
        y = np.dot(W, x)

        # Error
        error = t - y

        # Delta rule update
        W = W + eta * error * x
```

```
            total_error += error**2

        if epoch % 10 == 0:
            print(f"Epoch {epoch}, Error: {total_error:.4f}")

print("Final weights:", W)
```

Output

Sample output (values may vary due to random initialization):

```
Epoch 0, Error: 1.3702
Epoch 10, Error: 0.3323
Epoch 20, Error: 0.1067
Epoch 30, Error: 0.0341
Epoch 40, Error: 0.0109
Final weights: [-0.77979452  0.30031434  1.11145604]

Final weights: [0.85, -0.32, 0.67]
```

Conclusion

The Delta Rule successfully adjusts the weights of the neuron to reduce the error over time. As the number of epochs increases, the total error decreases, showing that the model is learning.